PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Laboratory — Lab 5: Static Code Analysis

Restaurant Dining Order Billing & Kitchen Ticket System (#49)

Student Name: Subramani B M	**SRN:** PES1UG24CS473	**Section:** H
Assigned Repo: SETAPESU26/L5_49_restaurant_order_system	**Personal Fork:** SubramaniBM/L5_49_restaurant_order_system	**Final Pylint Score: 10.00 / 10.00 (Perfect)**

Executive Summary & Static Analysis Objectives:

In this laboratory, industry-standard static analysis tools (**Pylint**, **Bandit**, and **Flake8**) were deployed to inspect, document, and remediate all security vulnerabilities, code smells, design flaws, and PEP 8 style violations in restaurant_order_system.py. All identified high/medium security issues were eliminated, formatting violations resolved, and code quality elevated from an initial 4.81/10 to a perfect **10.00/10.00**.

Static Analysis Tool	Baseline (Before Fixes)	Remediated (After Fixes)	Quality Delta & Status
Pylint Code Quality	4.81 / 10.00 (22 Warnings/Conventions)	10.00 / 10.00 (Zero Defects)	**+5.19 (+107.9%) — PERFECT ✓**
Bandit Security Scanner	5 Issues (2 High, 1 Medium, 2 Low)	0 Issues Identified	**100% Security Vulnerability Remediation ✓**
Flake8 Style Enforcer	9 PEP 8 Style & Formatting Violations	0 Violations	**100% PEP 8 Compliance ✓**

Documented Issues & Remediation Table (High/Medium Severity Prioritized)

#	Code	Tool	Severity	Line	Description of Issue	Applied Remediation Strategy
1	B605	Bandit	High	58	os.system() subshell call allows shell injection vulnerability.	Replaced with safe Python print(), eliminating subshell execution.
2	B324	Bandit	High	47	Weak MD5 hash algorithm used for digital kitchen ticket verification.	Upgraded to cryptographically secure, collision-resistant SHA-256.
3	B307	Bandit	Medium	42	Insecure eval() on promo expression allows remote code execution.	Removed eval(); implemented safe arithmetic parsing for multiplier.
4	B311	Bandit	Low	51	Standard pseudo-random generator used for POS authorization PIN.	Replaced random with secrets.SystemRandom() cryptographic entropy.
5	B110	Bandit	Low	26	Bare try-except-pass silently suppresses all unexpected exceptions.	Replaced with explicit membership check if table_id in orders:.
6	W0102	Pylint	Warning	15	Mutable default argument order_log=[] shared across function calls.	Set default to None and initialized order_log = [] inside function body.
7	C0103	Pylint	Conv	15..40	CamelCase function names violate PEP 8 snake_case style.	Renamed to place_table_order, void_table_order, check_table_status.
8	C0116	Pylint	Conv	15..61	Missing function docstrings across 8 module functions.	Added descriptive docstrings detailing arguments, purpose, and return.
9	R1732	Pylint	Refactor	55	open() called without context manager and explicit UTF-8 encoding.	Refactored to with open(filename, "w", encoding="utf-8") as f:.
10	W0621	Pylint	Warning	61	Parameter orders shadows global dictionary; unused key variable.	Renamed parameter orders_dict; iterated values directly via .values().
11	E711	Flake8	Style	30	Comparison orders.get(table_id) == None violates singleton standard.	Updated to idiomatic identity check if orders.get(table_id) is None:.
12	F401	Flake8	Import	4, 6	Unused imports os and math waste namespace memory overhead.	Removed redundant imports os and math completely from module.
13	E501	Flake8	Style	18, 19	Lines exceed PEP 8 79-character limit (e.g. line 18 had 110 chars).	Reformatted dictionary mappings and function calls across indented lines.
14	W292	Flake8	Style	83	Missing terminating newline character at end of file.	Appended POSIX-compliant trailing newline character ( ).

Comprehensive Reflection & Engineering Analysis

1. Static Code Analysis in Enterprise Software Quality & Maintainability:

Static code analysis inspects source code without execution by analyzing the Abstract Syntax Tree (AST). It provides automated quality gating that uncovers subtle bugs, vulnerabilities, and architectural anti-patterns long before code reaches deployment.

• **Cost Efficiency:** Resolving vulnerabilities (e.g., eval() or shell command execution) during static analysis is orders of magnitude less expensive than addressing security breaches in production environments.

• **Eliminating Silent Failures:** Anti-patterns such as mutable default arguments (order_log=[]) do not crash code on initial testing but corrupt shared state in concurrent environments. Static analyzers detect these immediately.

• **Team Standardization:** Enforcing PEP 8 styling, naming conventions, and docstrings creates uniform readability and drastically lowers cognitive load during code reviews.

2. Comparative Tool Synergy (Pylint vs. Bandit vs. Flake8):

Each static analysis tool is specialized for a distinct facet of software quality:

• **Bandit:** Specialized AST security scanner identifying CWE vulnerabilities (command injection, weak cryptography, insecure randomness).

• **Flake8:** Ultra-fast linter combining PyFlakes, pycodestyle, and McCabe to enforce PEP 8 style, line length limits (≤79 chars), and unused imports.

• **Pylint:** Comprehensive semantic code reviewer evaluating design patterns, scope shadowing, mutable defaults, and scoring overall quality out of 10.0.

Defense-in-Depth: Relying solely on Flake8 leaves security holes untouched; relying solely on Bandit ignores code smells and resource leaks. Combining all three provides holistic coverage across security, style, and software design.

3. Severity-Based Triage & CI/CD Pipeline Strategy:

In modern CI/CD pipelines, findings must be triaged by severity to balance security posture against developer velocity:

• **Tier 1 — High & Medium Security Vulnerabilities (Bandit):** Immediate build failure (Blocker). No code can merge until arbitrary execution (eval) or shell injections are eliminated.

• **Tier 2 — Logic & Resource Warnings (Pylint):** PR review blocker. Unclosed files or mutable defaults must be resolved within the active development sprint.

• **Tier 3 — Style & Formatting (Flake8):** Automated via git pre-commit hooks and auto-formatters (e.g., black, isort) to eliminate manual reviewer overhead.

4. Deep Dive into Dangerous Anti-Patterns (Mutable Defaults & eval):

• **Mutable Default Argument (order_log=[]):** In Python, default parameter objects are evaluated and bound once when the function definition is executed at module load time. Successive invocations of place_table_order that omit an explicit list append to the exact same list in memory. In a restaurant system, this causes Table 4 to inherit the order logs and dishes of Table 1! Fixed via sentinel default order_log=None.

• **Arbitrary String Evaluation (eval(promo_expr)):** Allowing user-supplied expressions to be executed through eval() gives attackers full remote code execution capabilities (e.g. __import__("os").system(...)). Replaced with strict tokenized arithmetic evaluation.

Git Commits & Verification on Personal Fork:

Three disciplined commits were pushed to **https://github.com/SubramaniBM/L5_49_restaurant_order_system**:

1. da707ce: *Fix Bandit security issues: eliminate eval, replace os.system shell call, use sha256 and secure random pin*
2. 1f5108b: *Fix Flake8 PEP 8 style issues: remove unused imports, wrap long lines, fix None comparison, and add trailing newline*
3. b4583db: *Fix Pylint code quality issues, achieve 10.00/10 score, add reports, issues table, and reflection document*